

Đây là notebook đi kèm cho cuốn sách [Deep Learning with Python, Third Edition](https://www.manning.com/books/deep-learning-with-
**Nếu bạn muốn theo dõi chi tiết nội dung, tôi khuyên bạn nên đọc notebook này song song với bản in hoặc bản điện tử của cuốn sách
Nội dung cuốn sách có sẵn trực tuyến tại deeplearningwithpython.io.

```
!pip install keras keras-hub --upgrade -q
```

```
import os  
os.environ["KERAS_BACKEND"] = "jax"
```

Hiện mã

```
## Sinh ảnh (Image generation)
```

```
### Deep learning cho việc sinh ảnh
```

```
#### Lấy mẫu từ không gian tiềm ẩn (latent spaces) của hình ảnh
```

```
#### Variational autoencoders (VAE)
```

```
#### Triển khai một VAE với Keras
```

```
import keras  
from keras import layers  
  
latent_dim = 2  
  
image_inputs = keras.Input(shape=(28, 28, 1))  
x = layers.Conv2D(32, 3, activation="relu", strides=2, padding="same")(image_inputs)  
x = layers.Conv2D(64, 3, activation="relu", strides=2, padding="same")(x)  
x = layers.Flatten()(x)  
x = layers.Dense(16, activation="relu")(x)  
z_mean = layers.Dense(latent_dim, name="z_mean")(x)  
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)  
encoder = keras.Model(image_inputs, [z_mean, z_log_var], name="encoder")
```

```
encoder.summary(line_length=80)
```

```
from keras import ops  
  
class Sampler(keras.Layer):  
    def __init__(self, **kwargs):  
        super().__init__(**kwargs)  
        self.seed_generator = keras.random.SeedGenerator()  
        self.built = True  
  
    def call(self, z_mean, z_log_var):  
        batch_size = ops.shape(z_mean)[0]  
        z_size = ops.shape(z_mean)[1]  
        epsilon = keras.random.normal(  
            (batch_size, z_size), seed=self.seed_generator  
        )  
        return z_mean + ops.exp(0.5 * z_log_var) * epsilon
```

```

latent_inputs = keras.Input(shape=(latent_dim,))
x = layers.Dense(7 * 7 * 64, activation="relu")(latent_inputs)
x = layers.Reshape((7, 7, 64))(x)
x = layers.Conv2DTranspose(64, 3, activation="relu", strides=2, padding="same")(
    x
)
x = layers.Conv2DTranspose(32, 3, activation="relu", strides=2, padding="same")(
    x
)
decoder_outputs = layers.Conv2D(1, 3, activation="sigmoid", padding="same")(x)
decoder = keras.Model(latent_inputs, decoder_outputs, name="decoder")

```

```

decoder.summary(line_length=80)

```

```

class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.sampler = Sampler()
        self.reconstruction_loss_tracker = keras.metrics.Mean(
            name="reconstruction_loss"
        )
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")

    def call(self, inputs):
        return self.encoder(inputs)

    def compute_loss(self, x, y, y_pred, sample_weight=None, training=True):
        original = x
        z_mean, z_log_var = y_pred
        reconstruction = self.decoder(self.sampler(z_mean, z_log_var))

        reconstruction_loss = ops.mean(
            ops.sum(
                keras.losses.binary_crossentropy(x, reconstruction), axis=(1, 2)
            )
        )
        kl_loss = -0.5 * (
            1 + z_log_var - ops.square(z_mean) - ops.exp(z_log_var)
        )
        total_loss = reconstruction_loss + ops.mean(kl_loss)

        self.reconstruction_loss_tracker.update_state(reconstruction_loss)
        self.kl_loss_tracker.update_state(kl_loss)
        return total_loss

```

```

import numpy as np

(x_train, _), (x_test, _) = keras.datasets.mnist.load_data()
mnist_digits = np.concatenate([x_train, x_test], axis=0)
mnist_digits = np.expand_dims(mnist_digits, -1).astype("float32") / 255

vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam())
vae.fit(mnist_digits, epochs=30, batch_size=128)

```

```

import matplotlib.pyplot as plt

n = 30
digit_size = 28
figure = np.zeros((digit_size * n, digit_size * n))

grid_x = np.linspace(-1, 1, n)
grid_y = np.linspace(-1, 1, n)[::-1]

for i, yi in enumerate(grid_y):
    for j, xi in enumerate(grid_x):
        z_sample = np.array([[xi, yi]])
        x_decoded = vae.decoder.predict(z_sample)
        digit = x_decoded[0].reshape(digit_size, digit_size)
        figure[
            i * digit_size : (i + 1) * digit_size,

```

```

        j * digit_size : (j + 1) * digit_size,
    ] = digit

plt.figure(figsize=(15, 15))
start_range = digit_size // 2
end_range = n * digit_size + start_range
pixel_range = np.arange(start_range, end_range, digit_size)
sample_range_x = np.round(grid_x, 1)
sample_range_y = np.round(grid_y, 1)
plt.xticks(pixel_range, sample_range_x)
plt.yticks(pixel_range, sample_range_y)
plt.xlabel("z[0]")
plt.ylabel("z[1]")
plt.axis("off")
plt.imshow(figure, cmap="Greys_r")

```

Mô hình khuếch tán (Diffusion models)

Tập dữ liệu Oxford Flowers

```

import os

fpath = keras.utils.get_file(
    origin="https://www.robots.ox.ac.uk/~vgg/data/flowers/102/102flowers.tgz",
    extract=True,
)

```

```

batch_size = 32
image_size = 128
images_dir = os.path.join(fpath, "jpg")
dataset = keras.utils.image_dataset_from_directory(
    images_dir,
    labels=None,
    image_size=(image_size, image_size),
    crop_to_aspect_ratio=True,
)
dataset = dataset.rebatch(
    batch_size,
    drop_remainder=True,
)

```

```

from matplotlib import pyplot as plt

for batch in dataset:
    img = batch.numpy()[0]
    break
plt.imshow(img.astype("uint8"))

```

Một U-Net denoising autoencoder

```

def residual_block(x, width):
    input_width = x.shape[3]
    if input_width == width:
        residual = x
    else:
        residual = layers.Conv2D(width, 1)(x)
    x = layers.BatchNormalization(center=False, scale=False)(x)
    x = layers.Conv2D(width, 3, padding="same", activation="swish")(x)
    x = layers.Conv2D(width, 3, padding="same")(x)
    x = x + residual
    return x

def get_model(image_size, widths, block_depth):
    noisy_images = keras.Input(shape=(image_size, image_size, 3))
    noise_rates = keras.Input(shape=(1, 1, 1))

    x = layers.Conv2D(widths[0], 1)(noisy_images)

```

```

n = layers.UpSampling2D(image_size, interpolation="nearest")(noise_rates)
x = layers.Concatenate()([x, n])

skips = []
for width in widths[:-1]:
    for _ in range(block_depth):
        x = residual_block(x, width)
        skips.append(x)
    x = layers.AveragePooling2D(pool_size=2)(x)

for _ in range(block_depth):
    x = residual_block(x, widths[-1])

for width in reversed(widths[:-1]):
    x = layers.UpSampling2D(size=2, interpolation="bilinear")(x)
    for _ in range(block_depth):
        x = layers.Concatenate()([x, skips.pop()])
        x = residual_block(x, width)

pred_noise_masks = layers.Conv2D(3, 1, kernel_initializer="zeros")(x)

return keras.Model([noisy_images, noise_rates], pred_noise_masks)

```

Các khái niệm về thời gian khuếch tán (diffusion time) và lịch trình khuếch tán (diffusion schedule)

```

def diffusion_schedule(
    diffusion_times,
    min_signal_rate=0.02,
    max_signal_rate=0.95,
):
    start_angle = ops.cast(ops.arccos(max_signal_rate), "float32")
    end_angle = ops.cast(ops.arccos(min_signal_rate), "float32")
    diffusion_angles = start_angle + diffusion_times * (end_angle - start_angle)
    signal_rates = ops.cos(diffusion_angles)
    noise_rates = ops.sin(diffusion_angles)
    return noise_rates, signal_rates

```

```

diffusion_times = ops.arange(0.0, 1.0, 0.01)
noise_rates, signal_rates = diffusion_schedule(diffusion_times)

diffusion_times = ops.convert_to_numpy(diffusion_times)
noise_rates = ops.convert_to_numpy(noise_rates)
signal_rates = ops.convert_to_numpy(signal_rates)

plt.plot(diffusion_times, noise_rates, label="Noise rate")
plt.plot(diffusion_times, signal_rates, label="Signal rate")

plt.xlabel("Diffusion time")
plt.legend()

```

Quy trình huấn luyện

```

class DiffusionModel(keras.Model):
    def __init__(self, image_size, widths, block_depth, **kwargs):
        super().__init__(**kwargs)
        self.image_size = image_size
        self.denoising_model = get_model(image_size, widths, block_depth)
        self.seed_generator = keras.random.SeedGenerator()
        self.loss = keras.losses.MeanAbsoluteError()
        self.normalizer = keras.layers.Normalization()

    def denoise(self, noisy_images, noise_rates, signal_rates):
        pred_noise_masks = self.denoising_model([noisy_images, noise_rates])
        pred_images = (
            noisy_images - noise_rates * pred_noise_masks
        ) / signal_rates
        return pred_images, pred_noise_masks

    def call(self, images):
        images = self.normalizer(images)

```

```

noise_masks = keras.random.normal(
    (batch_size, self.image_size, self.image_size, 3),
    seed=self.seed_generator,
)
diffusion_times = keras.random.uniform(
    (batch_size, 1, 1, 1),
    minval=0.0,
    maxval=1.0,
    seed=self.seed_generator,
)
noise_rates, signal_rates = diffusion_schedule(diffusion_times)
noisy_images = signal_rates * images + noise_rates * noise_masks
pred_images, pred_noise_masks = self.denoise(
    noisy_images, noise_rates, signal_rates
)
return pred_images, pred_noise_masks, noise_masks

def compute_loss(self, x, y, y_pred, sample_weight=None, training=True):
    _, pred_noise_masks, noise_masks = y_pred
    return self.loss(noise_masks, pred_noise_masks)

def generate(self, num_images, diffusion_steps):
    noisy_images = keras.random.normal(
        (num_images, self.image_size, self.image_size, 3),
        seed=self.seed_generator,
    )
    step_size = 1.0 / diffusion_steps
    for step in range(diffusion_steps):
        diffusion_times = ops.ones((num_images, 1, 1, 1)) - step * step_size
        noise_rates, signal_rates = diffusion_schedule(diffusion_times)
        pred_images, pred_noises = self.denoise(
            noisy_images, noise_rates, signal_rates
        )
        next_diffusion_times = diffusion_times - step_size
        next_noise_rates, next_signal_rates = diffusion_schedule(
            next_diffusion_times
        )
        noisy_images = (
            next_signal_rates * pred_images + next_noise_rates * pred_noises
        )
    images = (
        self.normalizer.mean + pred_images * self.normalizer.variance**0.5
    )
    return ops.clip(images, 0.0, 255.0)

```

Quy trình khởi tạo/sinh ảnh

Trực quan hóa kết quả với một callback tùy chỉnh

```

class VisualizationCallback(keras.callbacks.Callback):
    def __init__(self, diffusion_steps=20, num_rows=3, num_cols=6):
        self.diffusion_steps = diffusion_steps
        self.num_rows = num_rows
        self.num_cols = num_cols

    def on_epoch_end(self, epoch=None, logs=None):
        generated_images = self.model.generate(
            num_images=self.num_rows * self.num_cols,
            diffusion_steps=self.diffusion_steps,
        )

        plt.figure(figsize=(self.num_cols * 2.0, self.num_rows * 2.0))
        for row in range(self.num_rows):
            for col in range(self.num_cols):
                i = row * self.num_cols + col
                plt.subplot(self.num_rows, self.num_cols, i + 1)
                img = ops.convert_to_numpy(generated_images[i]).astype("uint8")
                plt.imshow(img)
                plt.axis("off")
        plt.tight_layout()
        plt.show()
        plt.close()

```

Bắt đầu huấn luyện!

```

model = DiffusionModel(image_size, widths=[32, 64, 96, 128], block_depth=2)
model.normalizer.adapt(dataset)

```

```

model.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=keras.optimizers.schedules.InverseTimeDecay(
            initial_learning_rate=1e-3,
            decay_steps=1000,
            decay_rate=0.1,
        ),
        use_ema=True,
        ema_overwrite_frequency=100,
    ),
)

```

```

model.fit(
    dataset,
    epochs=100,
    callbacks=[
        VisualizationCallback(),
        keras.callbacks.ModelCheckpoint(
            filepath="diffusion_model.weights.h5",
            save_weights_only=True,
            save_best_only=True,
        ),
    ],
)

```

Các mô hình Text-to-image (Chuyển văn bản thành hình ảnh)

```

if keras.config.backend() == "torch":
    # The rest of this chapter will not do any training. The following keeps
    # PyTorch from using too much memory by disabling gradients. TensorFlow and
    # JAX use a much smaller memory footprint and do not need this hack.
    import torch

    torch.set_grad_enabled(False)

```

```

import keras_hub

height, width = 512, 512
task = keras_hub.models.TextToImage.from_preset(
    "stable_diffusion_3_medium",

```

```

        image_shape=(height, width, 3),
        dtype="float16",
    )
    prompt = "A NASA astronaut riding an origami elephant in New York City"
    task.generate(prompt)

```

```

task.generate(
    {
        "prompts": prompt,
        "negative_prompts": "blue color",
    }
)

```

```

import numpy as np
from PIL import Image

def display(images):
    return Image.fromarray(np.concatenate(images, axis=1))

display([task.generate(prompt, num_steps=x) for x in [5, 10, 15, 20, 25]])

```

Khám phá không gian tiềm ẩn của một mô hình text-to-image

```

from keras import random

def get_text_embeddings(prompt):
    token_ids = task.preprocessor.generate_preprocess([prompt])
    negative_token_ids = task.preprocessor.generate_preprocess([""])
    return task.backbone.encode_text_step(token_ids, negative_token_ids)

def denoise_with_text_embeddings(embeddings, num_steps=28, guidance_scale=7.0):
    latents = random.normal((1, height // 8, width // 8, 16))
    for step in range(num_steps):
        latents = task.backbone.denoise_step(
            latents,
            embeddings,
            step,
            num_steps,
            guidance_scale,
        )
    return task.backbone.decode_step(latents)[0]

def scale_output(x):
    x = ops.convert_to_numpy(x)
    x = np.clip((x + 1.0) / 2.0, 0.0, 1.0)
    return np.round(x * 255.0).astype("uint8")

embeddings = get_text_embeddings(prompt)
image = denoise_with_text_embeddings(embeddings)
scale_output(image)

```

```
[x.shape for x in embeddings]
```

```

from keras import ops

def slerp(t, v1, v2):
    v1, v2 = ops.cast(v1, "float32"), ops.cast(v2, "float32")
    v1_norm = ops.linalg.norm(ops.ravel(v1))
    v2_norm = ops.linalg.norm(ops.ravel(v2))
    dot = ops.sum(v1 * v2 / (v1_norm * v2_norm))
    theta_0 = ops.arccos(dot)
    sin_theta_0 = ops.sin(theta_0)
    theta_t = theta_0 * t
    sin_theta_t = ops.sin(theta_t)
    s0 = ops.sin(theta_0 - theta_t) / sin_theta_0
    s1 = sin_theta_t / sin_theta_0
    return s0 * v1 + s1 * v2

def interpolate_text_embeddings(e1, e2, start=0, stop=1, num=10):
    embeddings = []
    for t in np.linspace(start, stop, num):

```

```
        embeddings.append(  
            (  
                slerp(t, e1[0], e2[0]),  
                e1[1],  
                slerp(t, e1[2], e2[2]),  
                e1[3],  
            )  
        )  
    )  
    return embeddings
```

```
prompt1 = "A friendly dog looking up in a field of flowers"  
prompt2 = "A horrifying, tentacled creature hovering over a field of flowers"  
e1 = get_text_embeddings(prompt1)  
e2 = get_text_embeddings(prompt2)
```

```
.    --
```